CS-3083 Project 6 Report

Bradley Titagwan

Decision Trees and Random Forest

Introduction and Task Overview

The purpose of this project was to implement and evaluate several machine learning classification algorithms completely from scratch using Python. The algorithms implemented in this project were the C4.5 Decision Tree algorithm, Random Forest, and AdaBoost. These algorithms were evaluated using two datasets: the MNIST handwritten digit dataset and the Breast Cancer Wisconsin dataset. The goals of the project were to understand recursive decision tree learning, entropy and gain ratio calculations, ensemble learning methods, and the effects of hyperparameter tuning on classification performance. Another major goal was to compare the strengths and weaknesses of individual decision trees versus ensemble methods such as Random Forest and AdaBoost. All algorithms were implemented manually without using machine learning libraries such as scikit-learn. Only standard Python libraries, NumPy, matplotlib, time, and OS were used throughout the project.

Datasets

MNIST Dataset

Two datasets were used during experimentation. The first dataset was the MNIST handwritten digit dataset. MNIST consists of grayscale handwritten digit images where each image is 28 × 28 pixels, resulting in 784 total features per image after flattening. Each image belongs to one of ten classes representing the digits 0 through 9. MNIST is considered a difficult classification problem because it contains very high-dimensional data, handwritten variation between digits, and overlapping visual patterns. To reduce runtime and allow practical experimentation, subsets containing 5000 training examples and 1000 testing examples were used.

Breast Cancer Wisconsin Dataset

The second dataset used was the Breast Cancer Wisconsin dataset obtained from Kaggle/UCI. This dataset is a binary classification problem used to predict whether a tumor is malignant or benign based on numerical tumor measurements. The first column of the dataset contained an ID value, the second column contained the diagnosis label, and all remaining columns contained feature values describing cell characteristics. The diagnosis labels were encoded such that malignant tumors were represented by 1 and benign tumors were represented by 0. The ID column was removed because it provided no predictive value. Compared to MNIST, this dataset was much easier because it contained fewer features and more structured numerical patterns.

Extra Dataset Link: Breast Cancer Wisconsin Dataset

Preprocessing

MNIST Preprocessing

Several preprocessing steps were required before training the models. For the MNIST dataset, the images were first loaded and flattened from 28 × 28 matrices into vectors containing 784 features. Pixel values were then binarized using a threshold of 128. Pixels greater than or equal to 128 were converted to 1, while pixels below 128 were converted to 0. Binarization simplified the splitting process because each feature only had two possible values. Without binarization, each pixel could contain 256 possible grayscale values, greatly increasing the complexity of the decision tree splits. Although binarization reduced computational complexity, it also removed grayscale detail that may have helped classification performance.

MNIST Preprocessing

For the Breast Cancer dataset, preprocessing involved removing the ID column, encoding diagnosis labels into binary values, converting all feature values into floating point numbers, and discretizing numerical features into bins. Discretization was necessary because the C4.5 implementation used categorical splitting methods. Converting continuous values into discrete bins allowed the decision tree to perform feature comparisons more efficiently.

Method of Implementation

C4.5 Decision Tree Implementation

Entropy was used to measure impurity within a node. Lower entropy indicated purer nodes, while higher entropy indicated mixed labels. The entropy formula used in the implementation was:

The C4.5 Decision Tree algorithm was implemented recursively using a custom TreeNode class. Each node stored the selected splitting feature, a dictionary of child nodes, a majority class prediction, and a flag indicating whether the node was a leaf node.

Python dictionaries were chosen to store child branches because they allow fast lookup, efficient branching, and a natural recursive representation of tree structures. The complete decision tree was represented through linked TreeNode objects. At each recursive step, the corrected implementation first calculated the information gain for every candidate feature. It then calculated the average positive information gain and eliminated features whose information gain was below that average

Gain ratio was calculated only for the remaining eligible features, and the feature with the highest gain ratio was selected for the split. This two-stage attribute-selection procedure follows C4.5 more closely and prevents a feature with very low information gain and very low split information from receiving an artificially high gain ratio.

After the best feature was selected, child branches were created for its observed values and recursion continued. Stopping conditions included reaching the maximum depth, having too few remaining samples, exhausting the available features, or reaching a node where all labels belonged to the same class.

$$\boldsymbol{H(S) = -\sum p(x) \log_2 p(x)}$$

Information gain measured the reduction in entropy created by a candidate split. The average-information-gain filter was applied before gain ratio so that only features providing at least an average reduction in uncertainty remained eligible.
Among those eligible features, C4.5 selected the feature with the highest gain ratio. Gain ratio divides information gain by split information, reducing bias toward attributes that create many small branches. The gain ratio formula used was:

$$\boldsymbol{GainRatio = \frac{InformationGain}{SplitInfo}}$$

Testing a New Example Using the C4.5 Tree

To classify a new example using the decision tree, prediction began at the root node. The algorithm checked the splitting feature at the current node, examined the feature value from the example, and followed the matching branch recursively until a leaf node was reached. The prediction stored in the leaf node was then returned as the final classification. If an unseen feature value appeared during testing, the majority class prediction stored at the node was returned to prevent failures.

Random Forest Implementation

Random Forest was implemented as an ensemble of multiple C4.5 decision trees. The forest stored all trees in a Python list because trees were independent and majority voting could be efficiently performed using list iteration. Each tree was trained using a bootstrap sample generated through random sampling with replacement. Random feature subsets were also selected at each split. These two sources of randomness reduced correlation between trees and improved generalization performance. During prediction, every tree generated a class prediction, and majority voting determined the final output.

AdaBoost Implementation

AdaBoost was implemented using decision stumps as weak learners. Each decision stump used one feature and one threshold to produce a single split. Initially, all training examples were assigned equal weights. After each iteration, misclassified examples received larger weights while correctly classified examples received smaller weights. This forced future weak learners to focus on difficult examples. AdaBoost assigned weights to each weak learner using the formula:

$$\alpha_t = \frac{1}{2}\ln\left(\frac{1 - error_t}{error_t}\right)$$

Because AdaBoost is naturally a binary classifier, a One-vs-Rest strategy was implemented for MNIST. Ten separate AdaBoost models were trained, where each model classified one digit against all other digits. During prediction, all models generated confidence scores, and the class with the highest score was selected as the final prediction.

Experimental Setup

For the Breast Cancer dataset, an 80/20 train-test split was used. Random Forest experiments commonly used 25 trees with a maximum depth of 10, while AdaBoost experiments commonly used 20 estimators. For the MNIST dataset, 40000 training examples and 5000 testing examples were used due to runtime limitations. Random Forest experiments used 100 trees, while AdaBoost experiments used 100 estimators by default.

Experimental Results and Comparisons

The Breast Cancer dataset produced excellent classification performance across all algorithms. The C4.5 decision tree achieved approximately 95.61% accuracy, while Random Forest achieved the highest overall performance at 98.25% accuracy. AdaBoost also performed extremely well, reaching 97.37% accuracy while maintaining significantly lower runtime than Random Forest. These results demonstrated the effectiveness of ensemble learning methods on structured tabular datasets.

Max Depth Experiments

Experiments involving maximum tree depth revealed important trends. Shallow trees underfit the data because they could not capture enough structure. Increasing tree depth improved performance initially, but accuracy eventually plateaued after a certain depth. This demonstrated diminishing returns from increasing tree complexity. Similar behavior occurred when varying the number of trees in Random Forest. Accuracy improved

Max Depth	2	4	6	8	10	12
Accuracy	93.86%	97.37%	97.37%	97.37%	97.37%	97.37%

significantly as the number of trees increased initially, but performance stabilized after approximately 10 trees while runtime continued increasing.

Random Feature Experiments

Experiments involving random feature selection revealed that increasing the number of candidate features improved split quality on the Breast Cancer dataset. However, excessively large feature subsets reduce randomness between trees and increase correlation between ensemble members.

Random Features	2	4	6	8	10
Accuracy	96.49%	97.37%	96.49%	97.37%	98.25%

AdaBoost Estimator Experiments

AdaBoost experiments demonstrated that only a small number of estimators were necessary to achieve strong performance on the Breast Cancer dataset. This suggested that boosting quickly captured useful classification patterns on simpler datasets.

Number Of Estimators	5	10	20	30	50
Accuracy	97.37%	96.49%	96.49%	96.49%	96.49%

Final MNIST Full Algorithm Comparison

The MNIST dataset proved substantially more difficult than the Breast Cancer dataset because of its high dimensionality and handwritten variation. After extensive tuning and experimentation, AdaBoost and Random Forest achieved dramatically improved performance.

These final results demonstrated the major performance improvements achieved through hyperparameter tuning and ensemble learning. Both Random Forest and AdaBoost significantly outperformed a single decision tree on high-dimensional handwritten image data.

Algorithm	C4.5 Decision Tree	Random Forest	AdaBoost One-vs-Rest
Accuracy	83.14%	94.84%	84.38%
Runtime	335.81 sec	1208.45 sec	510.45 sec

MNIST C4.5 Analysis

The C4.5 decision tree struggled on MNIST because handwritten digit recognition is a highly complex classification problem with 784 input features. Although the tree was able to learn some meaningful digit structures, the model still suffered from underfitting and limited generalization.

The confusion matrix revealed several major classification weaknesses.

True Digit	**0**	**1**	**3**	**7**	**9**
Common Incorrect Predictions	**4**	**4**	**1 and 4**	**4**	**4**

The model frequently predicted the digit 4 incorrectly because many handwritten digits shared overlapping visual structures. Digits such as 9, 7, and 0 often contained visual similarities that confused the decision tree.

AdaBoost One-vs-Rest Analysis

AdaBoost One-vs-Rest achieved the strongest overall MNIST performance at 84.38% accuracy. This was a major improvement over the single C4.5 tree because AdaBoost iteratively focused on difficult examples and corrected mistakes over multiple boosting rounds.

The final AdaBoost implementation used:

- 50 estimators
- One-vs-Rest multi-class classification
- sequential weighted learning

The confusion matrix showed that AdaBoost classified most digits very accurately.

Digit	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**
Correct Predictions	**433**	**554**	**419**	**414**	**441**	**346**	**406**	**423**	**365**	**418**

The strongest-performing digits were 1, 4 ,7 and 9.

The weakest-performing digits were 5 and 8

These digits were more difficult because handwritten versions often overlap visually with other classes.

Runtime Analysis

Runtime analysis revealed important tradeoffs between computational cost and predictive performance.

Algorithm	**Runtime**	**Observation**
C4.5	Lowest	Fast but weakest accuracy
Random Forest	Highest	Strong accuracy but expensive runtime
AdaBoost	Moderate	Best accuracy/runtime tradeoff

The C4.5 decision tree trained relatively quickly because only a single tree was constructed. Random Forest required significantly more runtime because many deep trees were trained independently. AdaBoost also required substantial runtime because each boosting iteration depended on the results of previous weak learners.

Interesting Discoveries

Several important discoveries were observed during experimentation. AdaBoost dramatically outperformed Random Forest and C4.5 on the MNIST dataset, suggesting that boosting handled difficult handwritten classification boundaries more effectively than bagging-based methods. Random Forest performed extremely well on structured tabular data such as the Breast Cancer dataset. Another important discovery was that increasing tree depth or the number of trees eventually produced diminishing returns. Additionally, smaller random feature subsets improved MNIST performance because excessive feature counts caused trees to become too correlated and overfit the data.

Discussion

This project demonstrated major differences between single-tree learning, bagging methods, and boosting methods. On the Breast Cancer dataset, all algorithms performed strongly, but ensemble methods clearly achieved the best results. Random Forest achieved the highest overall accuracy, while AdaBoost provided nearly identical performance with lower runtime. On the MNIST dataset, traditional tree methods struggled significantly because of the high-dimensional feature space and handwritten variation. AdaBoost dramatically outperformed the alternatives because boosting iteratively focused on difficult examples and created more adaptive decision boundaries.

The project also demonstrated the importance of hyperparameter tuning. Parameters such as tree depth, number of trees, random feature count, and number of AdaBoost estimators all significantly affected performance. The experiments highlighted the importance of balancing model complexity, runtime, and generalization performance.

Conclusion

This project successfully implemented C4.5 Decision Trees, Random Forest, and AdaBoost completely from scratch using Python. The experiments demonstrated that ensemble learning methods significantly improved performance over single decision trees. Random Forest achieved the highest overall Breast Cancer accuracy at 98.25%, while AdaBoost One-vs-Rest achieved the strongest MNIST performance at 73.1% accuracy.

The project provided a strong practical understanding of entropy, gain ratio, recursive tree construction, bagging, boosting, and ensemble learning methods. It also demonstrated the effects of dataset complexity, runtime tradeoffs, and hyperparameter tuning on machine learning performance.